

TRPO

LAMDA-5

April 28, 2026

我们这次的目标是实现算法 TRPO (Trust Region Policy Optimization, <https://arxiv.org/pdf/1502.05477>), 是 2015 年由当时在 UC Berkeley 读博的 John Schulman 及其导师 Pieter Abbeel 等人所提出, 发表在 JMLR, 目前引用量已过万。它成功地将传统运筹学中的“信赖域”方法引入深度强化学习, 首次在深度神经网络模型上为策略优化提供了“单调改进”的数学保证, 这意味着原则上每次更新都会让策略变得更好, 不会出现灾难性退步。

1 代码补全

该项目的实现参考了<https://spinningup.openai.com/en/latest/algorithms/trpo.html>, 请补全文件/algorithm/trpo.py 中的 TODO 部分 (完善函数 compute_advantages_from_traj, compute_advantages_from_rollout, 以及 _update_policy, 相关功能要求已在注释中标出)。并回答下述问题:

1.1 collect_trajectories 实现部分

1. collect_trajectories 与普通 rollout 采样方式相比, 数据收集的停止条件分别是什么?
2. 在 trajectory 采样模式下, 每一条轨迹需要保存哪些信息? 这些信息分别在后续 TRPO 更新中起什么作用?, 当环境返回终止信号时, 轨迹又会如何结束? 未达到最大长度的部分又应如何处理?
3. TrajectoryRollout 中的 masks 变量有什么作用? 如果不使用 masks, 计算 return 或 advantage 时可能出现什么问题?
4. collect_trajectories 在收集完一批轨迹后为什么需要重新初始化环境状态?

1.2 _update_policy 实现部分

该部分理论主要依赖于 Trust Region Policy Optimization 中的 Theorem 1, 代码实现参考 <https://spinningup.openai.com/en/latest/algorithms/trpo.html> 中的 Algorithm 1:

Theorem 1. Let $\alpha = D_{TV}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$. Then the following bound holds:

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2}\alpha^2$$

where $\epsilon = \max_{s,a} |A_{\pi}(s,a)|$ (8)

1. 解释该公式中每一项的实际意义, 这个公式在什么情况下可以保证策略的单调改进? 由此直观解释 TRPO 算法的设计思路 (优化目标和约束的设计? 先概述你的大致想法, 具体细节在后续提问中体现)。
2. 和文章 Approximately optimal approximate reinforcement learning 中的 Theorem 4.1 (即 TRPO 论文中的公式 (6)) 作对比, TRPO 作者在分析上的处理技术有什么具体的不同, 使得 TRPO 可以推广到所有一般的随机策略类而不是给定的混合策略类?
3. TRPO 中策略更新的 surrogate objective $L_{\pi_{\text{old}}}(\pi_{\text{new}})$ 一项在实际算法中是如何设计的?
4. 为什么 TRPO 需要约束新旧策略之间的 KL divergence 而不是定理中的 TV 距离? 在实际算法中我们采用 KL 散度的二阶近似 $D_{KL}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta}) \sim \frac{1}{2}\Delta\theta^T(\nabla_{\theta}^2 D_{KL})\Delta\theta$, 延续该思路解释为什么在 Algorithm 1 中我们会采用 line search 搜索参数 (Algorithm 1 line 8&9)?

2 探索部分

请从以下五个环境中至少选择三个进行测试: **Humanoid-v5**, **Ant-v5**, **HalfCheetah-v5**, **Hopper-v5**, **Walker2d-v5** 并思考下述问题:

1. 在相同超参数设置下, 三个环境的训练难度是否一致? 请结合 reward 曲线进行分析。
2. 哪个环境中 trajectory 与 rollout 的样本利用率差异最明显? 可能的原因是什么?
3. 在 trajectory 模式和 rollout 模式下, advantage 与 return 的计算方式有什么不同?

2.1 Advantage Normalization 的影响

1. 请分别设置 `advan_norm=true` 和 `advan_norm=false` 进行实验。两组实验的平均回报曲线有何差异?
2. 加入 advantage normalization 后, 训练过程是否更加稳定? 请结合 reward 曲线、KL divergence 或 surrogate objective 的变化进行分析。
3. 在你选择的测试环境中, advantage normalization 是否总是带来性能提升? 如果不是, 可能的原因是什么?

2.2 Trajectory 与 Rollout 的样本利用率比较

1. 在 rollout 模式下, 每次策略更新实际使用了多少个 transition? 在 trajectory 模式下, 每次策略更新又实际使用了多少个有效 transition?
2. 对比 trajectory 与 rollout 两种模式, 哪一种样本利用率更高?
3. 样本利用率更高是否一定意味着训练效果更好? 请结合实验曲线进行分析。

3 总结与反思

1. 在本项目中, `collect_trajectories` 的实现是否会改变 TRPO 的 on-policy 特性? 说出你的判断理由。
2. 如果只能保留一种采样方式, 你会选择 trajectory 还是 rollout? 请结合样本利用率和训练性能说明理由。
3. 完成实验报告的撰写后, 你对 TRPO 算法是否有了新的感悟或认识? (**这个问题不建议 AI 辅助生成, AI 时代最珍贵的是人类自我的想法。相信你写完报告后总会有一些自己的思考, 或许是疑惑或许是感悟, 完善与否不重要, 写出你自己的想法就好。**)
4. 针对本次算法实现, 你是否有什么想法和想改进的建议, 你在实现过程中有没有因为项目 README.md 或者实验报告引导和介绍问题导致卡了很久, 如果有, 请详细说明, 并给出你的建议。关于代码/实验报告问题难度你是否有感觉很难/简单?